

Physics-Informed Neural Network for Burgers' Equation

MATH 6201 Midterm Report

Isaac Adoboe - ieadoboe@mun.ca - 202384695

October 20, 2025

1 Introduction

Partial differential equations (PDEs) serve as the mathematical foundation for modeling countless physical phenomena, from fluid dynamics to heat transfer to quantum mechanics.

This project will ultimately solve Burgers' equation using PINNs (final report), but first we establish the classical numerical foundation. This report seeks to understand how PINNs can be used to solve the one-dimensional Burgers' partial differential equation (PDE), which combines advection and diffusion.

Despite its apparent simplicity, Burgers' equation captures the essential physics of more complex systems like the Navier-Stokes equations, making it an ideal testbed for numerical methods.

The numerical solution of partial differential equations (PDEs) has traditionally relied on discretization methods that partition a domain into grids. Finite difference, finite element, and finite volume methods have proven successful across many application domains.

These methods, however, are not without shortcomings. They require explicit grid generation, which can be computationally expensive for complex systems, and they struggle with higher-dimensional problems.

Physics-Informed Neural Networks (PINNs) offer a fundamentally different approach. By encoding the governing PDE into the loss function of a neural network, PINNs eliminate the need for spatial or temporal discretization in the traditional sense. The neural network serves as a continuous function approximator for the solution, with derivatives computed via automatic differentiation. This approach makes PINNs well suited for handling irregular domains and high-dimensional problems.

2 Why Burgers' Equation?

Burgers' equation (or Bateman–Burgers equation) is a fundamental partial differential equation and convection–diffusion equation appearing in various areas of applied mathematics, such as fluid mechanics, nonlinear acoustics, gas dynamics, traffic flow, and mathematical physics. [Wikipedia]

The equation was first introduced by Harry Bateman in 1915 and later studied by Johannes Martinus Burgers in 1948. For a given field $u(x, t)$ and diffusion coefficient (or kinematic viscosity, as in the original fluid mechanical context) ν , the general form of Burgers' equation (also known as the viscous Burgers' equation) in one space dimension is the dissipative system:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}. \quad (1)$$

This report establishes the theoretical and computational foundations for applying PINNs to Burgers' equation. We derive the governing equation, formulate the PINN loss function incorporating PDE residuals and initial conditions, and analyze training stability and convergence properties.

2.1 Problem Statement

We study the one-dimensional viscous Burgers' equation defined on a periodic spatial domain. This equation models the evolution of a velocity field $u(x, t)$ that experiences both nonlinear advection (self-transport) and viscous diffusion (smoothing). It is given by

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad (x, t) \in [0, 1] \times [0, T], \quad (2)$$

where $\nu > 0$ is the viscosity coefficient controlling the strength of diffusion. A smaller ν corresponds to weaker viscosity and thus sharper gradients in the solution.

The problem is defined with the following conditions:

$$\textbf{Initial condition:} \quad u(x, 0) = -\sin(\pi x), \quad x \in [0, 1], \quad (3)$$

$$\textbf{Periodic boundary conditions:} \quad u(0, t) = u(1, t), \quad \left. \frac{\partial u}{\partial x} \right|_{x=0} = \left. \frac{\partial u}{\partial x} \right|_{x=1}, \quad t \in [0, T]. \quad (4)$$

Here, the initial condition specifies that the velocity field starts as a negative sine wave, while the periodic boundaries make the spatial domain behave like a continuous loop values

at $x = 0$ and $x = 1$ are identical, as are their spatial derivatives.

For this study, we use a viscosity parameter of $\nu = 0.01$ and evolve the solution up to a final time of $T = 1.0$.

3 Derivation of Burgers' Equation

We derive it from Newton's second law applied to a fluid particle.

Consider a one-dimensional fluid flow where each particle has velocity $u(x, t)$ at position x and time t . In the absence of external forces and pressure gradients, the particle's acceleration must equal the viscous force per unit mass:

$$\frac{Du}{Dt} = \nu \frac{\partial^2 u}{\partial x^2} \quad (5)$$

where $\frac{D}{Dt}$ is the **material derivative** (rate of change following a fluid particle) and ν is the kinematic viscosity.

The material derivative in a fixed (Eulerian) reference frame expands as:

$$\frac{Du}{Dt} = \frac{\partial u}{\partial t} + \underbrace{u \frac{\partial u}{\partial x}}_{\text{advection term}} \quad (6)$$

The second term represents convection: a particle moving with velocity u experiences spatial gradients in u itself, leading to nonlinear self-interaction. Substituting into Newton's law yields:

$$\begin{aligned} (5) &= (6) \\ \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} &= \nu \frac{\partial^2 u}{\partial x^2} \end{aligned} \quad (7)$$

This is the standard form of the viscous Burgers' equation.

1. **Temporal rate of change** ($\partial u / \partial t$): Rate of change of velocity at a fixed point in space.
2. **Advective rate of change** ($u \partial u / \partial x$):
 - Responsible for shock formation (wave steepening)
 - If $u > 0$ and $\partial u / \partial x < 0$ (velocity decreasing downstream), faster fluid parcels overtake slower ones, creating compression waves
3. **Viscous diffusion** ($\nu \partial^2 u / \partial x^2$):

- Smooths out sharp gradients (opposes shock formation)
- For small ν , diffusion is weak, allowing quasi-shocks to develop

The interplay between these competing mechanisms (shock steepening and viscous smoothing) determines the structure of the solution.

Limiting Cases

- $\nu \rightarrow 0$: Reduces to Burgers' equation $u_t + uu_x = 0$, which admits discontinuous shock solutions.
- $u \rightarrow 0$: Becomes the heat equation $u_t = \nu u_{xx}$, whose solutions decay exponentially (dissipative).
- **Cole-Hopf transformation:** For initial-value problems, the substitution

$$u = -2\nu \frac{\partial}{\partial x} \ln \phi \quad (8)$$

transforms Burgers' equation into the linear heat equation for ϕ , enabling analytical solutions via Fourier series.

For our chosen initial condition $u(x, 0) = -\sin(\pi x)$ with periodic boundaries, an exact solution exists via Cole-Hopf, providing a benchmark for validating numerical methods.

4 Finite Difference Discretization

We discretize Burgers' equation using finite differences on a uniform grid, a classical approach that provides explicit control over error and stability.

Grid Generation

Partition the spatial domain $[0, 1]$ into N_x equally spaced points:

$$x_j = j\Delta x, \quad j = 0, 1, \dots, N_x - 1, \quad \Delta x = \frac{1}{N_x} \quad (9)$$

Partition the temporal domain $[0, T]$ into N_t steps:

$$t^n = n\Delta t, \quad n = 0, 1, \dots, N_t, \quad \Delta t = \frac{T}{N_t} \quad (10)$$

Let $u_j^n \approx u(x_j, t^n)$ denote the numerical approximation at grid point (x_j, t^n) .

4.1 Spatial Discretization

We approximate spatial derivatives using centered finite differences:

First Derivative (Advection Term)

$$\left. \frac{\partial u}{\partial x} \right|_j^n \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + \mathcal{O}(\Delta x^2) \quad (11)$$

This is a *second-order accurate* central difference. The $\mathcal{O}(\Delta x^2)$ error is derived via Taylor expansion:

$$u_{j+1}^n = u_j^n + \Delta x u_x|_j + \frac{\Delta x^2}{2} u_{xx}|_j + \frac{\Delta x^3}{6} u_{xxx}|_j + \dots \quad (12)$$

$$u_{j-1}^n = u_j^n - \Delta x u_x|_j + \frac{\Delta x^2}{2} u_{xx}|_j - \frac{\Delta x^3}{6} u_{xxx}|_j + \dots \quad (13)$$

Subtracting: $u_{j+1}^n - u_{j-1}^n = 2\Delta x u_x|_j + \mathcal{O}(\Delta x^3)$.

Second Derivative (Diffusion Term)

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_j^n \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} + \mathcal{O}(\Delta x^2) \quad (14)$$

Adding the Taylor expansions above yields: $u_{j+1}^n + u_{j-1}^n = 2u_j^n + \Delta x^2 u_{xx}|_j + \mathcal{O}(\Delta x^4)$.

4.2 Temporal Discretization

We use the **forward Euler** (explicit) method for time integration:

$$\left. \frac{\partial u}{\partial t} \right|_j^n \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} + \mathcal{O}(\Delta t) \quad (15)$$

This is *first-order accurate* in time. Higher-order methods (Runge-Kutta) will be considered for the final implementation if stability permits.

4.3 Fully Discretized Equation

Substituting the spatial and temporal approximations into Equation (2):

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} + u_j^n \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = \nu \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} \quad (16)$$

Rearranging to solve for u_j^{n+1} (explicit time-stepping):

Step 1: Multiply through by Δt to isolate u_j^{n+1} :

$$u_j^{n+1} - u_j^n + \Delta t \cdot u_j^n \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = \nu \Delta t \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} \quad (17)$$

Step 2: Move all terms except u_j^{n+1} to the right-hand side:

$$u_j^{n+1} = u_j^n - \Delta t \cdot u_j^n \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + \nu \Delta t \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} \quad (18)$$

Step 3: Simplify the advection term by combining constants:

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} u_j^n (u_{j+1}^n - u_{j-1}^n) + \frac{\nu \Delta t}{\Delta x^2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n) \quad (19)$$

This gives the final **explicit update formula**:

$$\boxed{u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} u_j^n (u_{j+1}^n - u_{j-1}^n) + \frac{\nu \Delta t}{\Delta x^2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n)} \quad (20)$$

Key observation: This is an *explicit* scheme. u_j^{n+1} depends only on values at the previous time level t^n . No linear system solve is required, but stability is conditionally guaranteed.

4.4 Boundary Condition Implementation

For periodic boundaries, we enforce:

$$u_0^n = u_{N_x}^n, \quad u_{-1}^n = u_{N_x-1}^n, \quad u_{N_x+1}^n = u_1^n \quad (21)$$

In practice, we use modular arithmetic for indices:

```
u[j+1] -> u[(j+1) % N_x]
u[j-1] -> u[(j-1 + N_x) % N_x]
```

4.5 Initial Condition

At $t = 0$, we set:

$$u_j^0 = -\sin(\pi x_j) = -\sin(\pi j \Delta x), \quad j = 0, 1, \dots, N_x - 1 \quad (22)$$

4.6 Algorithmic Summary

Algorithm 1 Explicit Finite Difference Solver for Burgers' Equation

```

1: Input:  $N_x, N_t, \nu, T$ 
2: Initialize:  $\Delta x = 1/N_x, \Delta t = T/N_t$ 
3: Set IC:  $u_j^0 = -\sin(\pi j \Delta x)$  for  $j = 0, \dots, N_x - 1$ 
4: for  $n = 0$  to  $N_t - 1$  do
5:   for  $j = 0$  to  $N_x - 1$  do
6:      $j_{\text{prev}} = (j - 1 + N_x) \bmod N_x$ 
7:      $j_{\text{next}} = (j + 1) \bmod N_x$ 
8:      $\text{advection} = -\frac{\Delta t}{2\Delta x} u_j^n (u_{j_{\text{next}}}^n - u_{j_{\text{prev}}}^n)$ 
9:      $\text{diffusion} = \frac{\nu \Delta t}{\Delta x^2} (u_{j_{\text{next}}}^n - 2u_j^n + u_{j_{\text{prev}}}^n)$ 
10:     $u_j^{n+1} = u_j^n + \text{advection} + \text{diffusion}$ 
11:   end for
12: end for
13: Output:  $u_j^n$  for all  $j, n$ 

```

5 Stability Analysis

Explicit finite difference schemes are conditionally stable—time step and grid spacing must satisfy certain constraints to prevent numerical instabilities. We perform von Neumann (Fourier) stability analysis to derive these conditions.

5.1 Von Neumann Stability Analysis

The method assumes perturbations grow or decay according to:

$$u_j^n = \xi^n e^{ikj\Delta x} \quad (23)$$

where k is the wavenumber and ξ is the *amplification factor*. Stability requires $|\xi| \leq 1$ for all k .

5.2 Linearized Analysis

Since Burgers' equation is nonlinear, we linearize around a constant background state $\bar{u}$:

$$u_j^n = \bar{u} + \epsilon_j^n, \quad |\epsilon| \ll 1 \quad (24)$$

The linearized equation becomes:

$$\frac{\partial \epsilon}{\partial t} + \bar{u} \frac{\partial \epsilon}{\partial x} = \nu \frac{\partial^2 \epsilon}{\partial x^2} \quad (25)$$

5.3 Application to Discretized Scheme

Substitute $\epsilon_j^n = \xi^n e^{ikj\Delta x}$ into the linearized version of Equation (20):

$$\xi^{n+1} e^{ikj\Delta x} = \xi^n e^{ikj\Delta x} - \frac{\bar{u}\Delta t}{2\Delta x} \xi^n (e^{ik(j+1)\Delta x} - e^{ik(j-1)\Delta x}) + \frac{\nu\Delta t}{\Delta x^2} \xi^n (e^{ik(j+1)\Delta x} - 2e^{ikj\Delta x} + e^{ik(j-1)\Delta x}) \quad (26)$$

Dividing by $\xi^n e^{ikj\Delta x}$:

$$\xi = 1 - \frac{\bar{u}\Delta t}{2\Delta x} (e^{ik\Delta x} - e^{-ik\Delta x}) + \frac{\nu\Delta t}{\Delta x^2} (e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \quad (27)$$

Using $e^{ik\Delta x} - e^{-ik\Delta x} = 2i \sin(k\Delta x)$ and $e^{ik\Delta x} + e^{-ik\Delta x} = 2 \cos(k\Delta x)$:

$$\xi = 1 - \frac{i\bar{u}\Delta t}{\Delta x} \sin(k\Delta x) + \frac{2\nu\Delta t}{\Delta x^2} (\cos(k\Delta x) - 1) \quad (28)$$

Using $\cos(\theta) - 1 = -2 \sin^2(\theta/2)$:

$$\xi = 1 - \frac{i\bar{u}\Delta t}{\Delta x} \sin(k\Delta x) - \frac{4\nu\Delta t}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) \quad (29)$$

5.4 Stability Condition

The magnitude of ξ is:

$$|\xi|^2 = \left[1 - \frac{4\nu\Delta t}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right)\right]^2 + \left[\frac{\bar{u}\Delta t}{\Delta x} \sin(k\Delta x)\right]^2 \quad (30)$$

For stability ($|\xi| \leq 1$), we require:

$$\boxed{\frac{4\nu\Delta t}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) \leq 1 \quad \text{and} \quad \left|\frac{\bar{u}\Delta t}{\Delta x}\right| \leq C} \quad (31)$$

where C is a constant of order unity. The worst case occurs at:

- **Diffusion stability:** $k\Delta x = \pi$ (highest frequency mode)

$$\frac{\nu\Delta t}{\Delta x^2} \leq \frac{1}{2} \quad \Rightarrow \quad \Delta t \leq \frac{\Delta x^2}{2\nu} \quad (32)$$

- **Advection stability (CFL condition):**

$$\Delta t \leq \frac{\Delta x}{|\bar{u}|_{\max}} \quad (33)$$

5.5 Combined Stability Criterion

For our nonlinear problem, $\bar{u}$ varies in space and time. We use the **local maximum velocity**:

$$\Delta t \leq \min \left(\frac{\Delta x}{|u|_{\max}}, \frac{\Delta x^2}{2\nu} \right) \quad (34)$$

5.6 Numerical Implications

For our parameters ($\nu = 0.01$, $|u|_{\max} \sim 1$ from IC):

- If $\Delta x = 0.01$ (100 grid points):

$$\Delta t_{\text{CFL}} \leq 0.01 \quad (35)$$

$$\Delta t_{\text{diffusion}} \leq \frac{(0.01)^2}{2(0.01)} = 0.005 \quad (\text{more restrictive}) \quad (36)$$

- The **diffusion constraint dominates** for small ν , severely limiting time step size.
- This is characteristic of *stiff* problems: explicit methods require prohibitively small Δt .

6 Conclusion

This report establishes the numerical foundation for solving the Burgers' equation. The final project will extend this work by implementing a Physics-Informed Neural Network as an alternative solution method. PINNs bypass grid generation and stability issues by encoding the PDE directly into a neural network loss function, leveraging automatic differentiation for derivative computation.

By comparing a well-established discretization technique with a modern machine learning method, this project aims to contribute to the ongoing dialogue between traditional numerical analysis and data-driven scientific computing.